

Taking a Sequence Length of 60

```
In [ ]: import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler
import tensorflow as tf
import joblib

SEQ_LEN = 60
FEATURES = ["Close"]
```

Dropping Nulls if any

```
In [ ]: df = pd.read_csv("gold_futures_full_history.csv")

if "Date" in df.columns:
    df["Date"] = pd.to_datetime(df["Date"])
    df = df.sort_values("Date").reset_index(drop=True)

df = df.dropna(subset=FEATURES).reset_index(drop=True)
```

```
In [ ]: data = df[FEATURES].astype("float32").values
```

Min-Max Scaling of the Features

```
In [ ]: scaler = MinMaxScaler()
data_scaled = scaler.fit_transform(data)
joblib.dump(scaler, "scaler_minmax.save")
```

```
Out[ ]: ['scaler_minmax.save']
```

Creating a Sequence length

```
In [ ]: def create_sequences(arr, seq_len):
    X, y = [], []
    for i in range(seq_len, len(arr)):
        X.append(arr[i-seq_len:i])
        y.append(arr[i])
```

```
return np.array(X), np.array(y)
```

```
X, y = create_sequences(data_scaled, SEQ_LEN)
```

```
In [ ]: total = len(X)
        train_end = int(total * 0.7)
        val_end = int(total * 0.85)

        X_train, y_train = X[:train_end], y[:train_end]
        X_val, y_val = X[train_end:val_end], y[train_end:val_end]
        X_test, y_test = X[val_end:], y[val_end:]
```

```
In [ ]: BATCH = 32

        train_ds = tf.data.Dataset.from_tensor_slices((X_train, y_train)).batch(BATCH).prefetch(tf.data.AUTOTUNE)
        val_ds = tf.data.Dataset.from_tensor_slices((X_val, y_val)).batch(BATCH).prefetch(tf.data.AUTOTUNE)
        test_ds = tf.data.Dataset.from_tensor_slices((X_test, y_test)).batch(BATCH).prefetch(tf.data.AUTOTUNE)
```

Building a LSTM Model

```
In [ ]: from tensorflow.keras.models import Sequential
        from tensorflow.keras.layers import LSTM, Dense, Dropout
        from tensorflow.keras.optimizers import Adam
        from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau

        num_features = X_train.shape[-1] # 1 for Close-only

        model = Sequential([
            LSTM(128, return_sequences=True, input_shape=(SEQ_LEN, num_features)),
            Dropout(0.2),

            LSTM(64),
            Dropout(0.2),

            Dense(num_features) # 1 output: next day Close
        ])

        model.compile(
            optimizer=Adam(0.001),
            loss="mse",
            metrics=["mae"]
```

```
)
model.summary()
```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/rnn/rnn.py:199: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```
super().__init__(**kwargs)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 60, 128)	66,560
dropout (Dropout)	(None, 60, 128)	0
lstm_1 (LSTM)	(None, 64)	49,408
dropout_1 (Dropout)	(None, 64)	0
dense (Dense)	(None, 1)	65

Total params: 116,033 (453.25 KB)

Trainable params: 116,033 (453.25 KB)

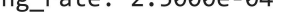
Non-trainable params: 0 (0.00 B)

Training the model

```
In [ ]: early_stop = EarlyStopping(monitor="val_loss", patience=6, restore_best_weights=True)
lr_sched = ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=3)

history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=80,
    callbacks=[early_stop, lr_sched],
    verbose=1
)
```

```

Epoch 1/80
27/27  10s 116ms/step - loss: 0.0010 - mae: 0.0265 - val_loss: 5.0478e-04 - val_mae: 0.0180 - learning_rate: 0.0010
Epoch 2/80
27/27  3s 103ms/step - loss: 9.8267e-04 - mae: 0.0252 - val_loss: 4.3668e-04 - val_mae: 0.0170 - learning_rate: 0.0010
Epoch 3/80
27/27  5s 105ms/step - loss: 0.0017 - mae: 0.0328 - val_loss: 0.0015 - val_mae: 0.0353 - learning_rate: 0.0010
Epoch 4/80
27/27  3s 103ms/step - loss: 0.0018 - mae: 0.0333 - val_loss: 3.7313e-04 - val_mae: 0.0152 - learning_rate: 0.0010
Epoch 5/80
27/27  3s 102ms/step - loss: 9.3451e-04 - mae: 0.0254 - val_loss: 3.6545e-04 - val_mae: 0.0157 - learning_rate: 0.0010
Epoch 6/80
27/27  6s 133ms/step - loss: 3.3753e-04 - mae: 0.0145 - val_loss: 7.6123e-04 - val_mae: 0.0244 - learning_rate: 0.0010
Epoch 7/80
27/27  4s 102ms/step - loss: 2.8132e-04 - mae: 0.0131 - val_loss: 3.4837e-04 - val_mae: 0.0149 - learning_rate: 0.0010
Epoch 8/80
27/27  3s 102ms/step - loss: 0.0012 - mae: 0.0288 - val_loss: 5.1133e-04 - val_mae: 0.0178 - learning_rate: 5.0000e-04
Epoch 9/80
27/27  4s 140ms/step - loss: 4.9899e-04 - mae: 0.0188 - val_loss: 2.4885e-04 - val_mae: 0.0120 - learning_rate: 5.0000e-04
Epoch 10/80
27/27  3s 119ms/step - loss: 2.3206e-04 - mae: 0.0119 - val_loss: 3.1356e-04 - val_mae: 0.0145 - learning_rate: 5.0000e-04
Epoch 11/80
27/27  3s 122ms/step - loss: 1.8239e-04 - mae: 0.0103 - val_loss: 3.0688e-04 - val_mae: 0.0143 - learning_rate: 5.0000e-04
Epoch 12/80
27/27  3s 104ms/step - loss: 4.5273e-04 - mae: 0.0176 - val_loss: 2.6473e-04 - val_mae: 0.0126 - learning_rate: 5.0000e-04
Epoch 13/80
27/27  4s 150ms/step - loss: 3.2588e-04 - mae: 0.0145 - val_loss: 2.2746e-04 - val_mae: 0.0122 - learning_rate: 2.5000e-04
Epoch 14/80
27/27  3s 103ms/step - loss: 1.7322e-04 - mae: 0.0101 - val_loss: 2.2091e-04 - val_mae: 0.0114 - learning_rate: 2.5000e-04

```

Epoch 15/80

27/27  3s 102ms/step - loss: 1.4098e-04 - mae: 0.0094 - val_loss: 2.1376e-04 - val_mae: 0.0117 - learning_rate: 2.5000e-04

Epoch 16/80

27/27  3s 105ms/step - loss: 1.4225e-04 - mae: 0.0089 - val_loss: 2.3380e-04 - val_mae: 0.0124 - learning_rate: 1.2500e-04

Epoch 17/80

27/27  4s 141ms/step - loss: 1.3883e-04 - mae: 0.0091 - val_loss: 2.2337e-04 - val_mae: 0.0121 - learning_rate: 1.2500e-04

Epoch 18/80

27/27  4s 101ms/step - loss: 1.4270e-04 - mae: 0.0090 - val_loss: 2.7750e-04 - val_mae: 0.0136 - learning_rate: 1.2500e-04

Epoch 19/80

27/27  3s 102ms/step - loss: 1.4388e-04 - mae: 0.0092 - val_loss: 3.3281e-04 - val_mae: 0.0151 - learning_rate: 6.2500e-05

Epoch 20/80

27/27  3s 101ms/step - loss: 1.3532e-04 - mae: 0.0090 - val_loss: 2.9262e-04 - val_mae: 0.0140 - learning_rate: 6.2500e-05

Epoch 21/80

27/27  5s 103ms/step - loss: 1.3770e-04 - mae: 0.0089 - val_loss: 2.5218e-04 - val_mae: 0.0129 - learning_rate: 6.2500e-05

Prediction using the Model

```
In [ ]: pred_scaled = model.predict(test_ds)
        pred_scaled = pred_scaled.reshape(-1, num_features)
```

6/6  1s 81ms/step

Inverse Transform the values to original ones

```
In [ ]: pred = scaler.inverse_transform(pred_scaled)
        y_test_orig = scaler.inverse_transform(y_test)
```

Metrics for measuring the performance of the model

```
In [ ]: from sklearn.metrics import mean_squared_error, mean_absolute_error
        import numpy as np

        rmse = np.sqrt(mean_squared_error(y_test_orig, pred))
        mae = mean_absolute_error(y_test_orig, pred)
```

```
mape = np.mean(np.abs((y_test_orig - pred) / y_test_orig)) * 100

print("LSTM RMSE:", rmse)
print("LSTM MAE:", mae)
print("LSTM MAPE:", mape)
```

LSTM RMSE: 81.15594014669382

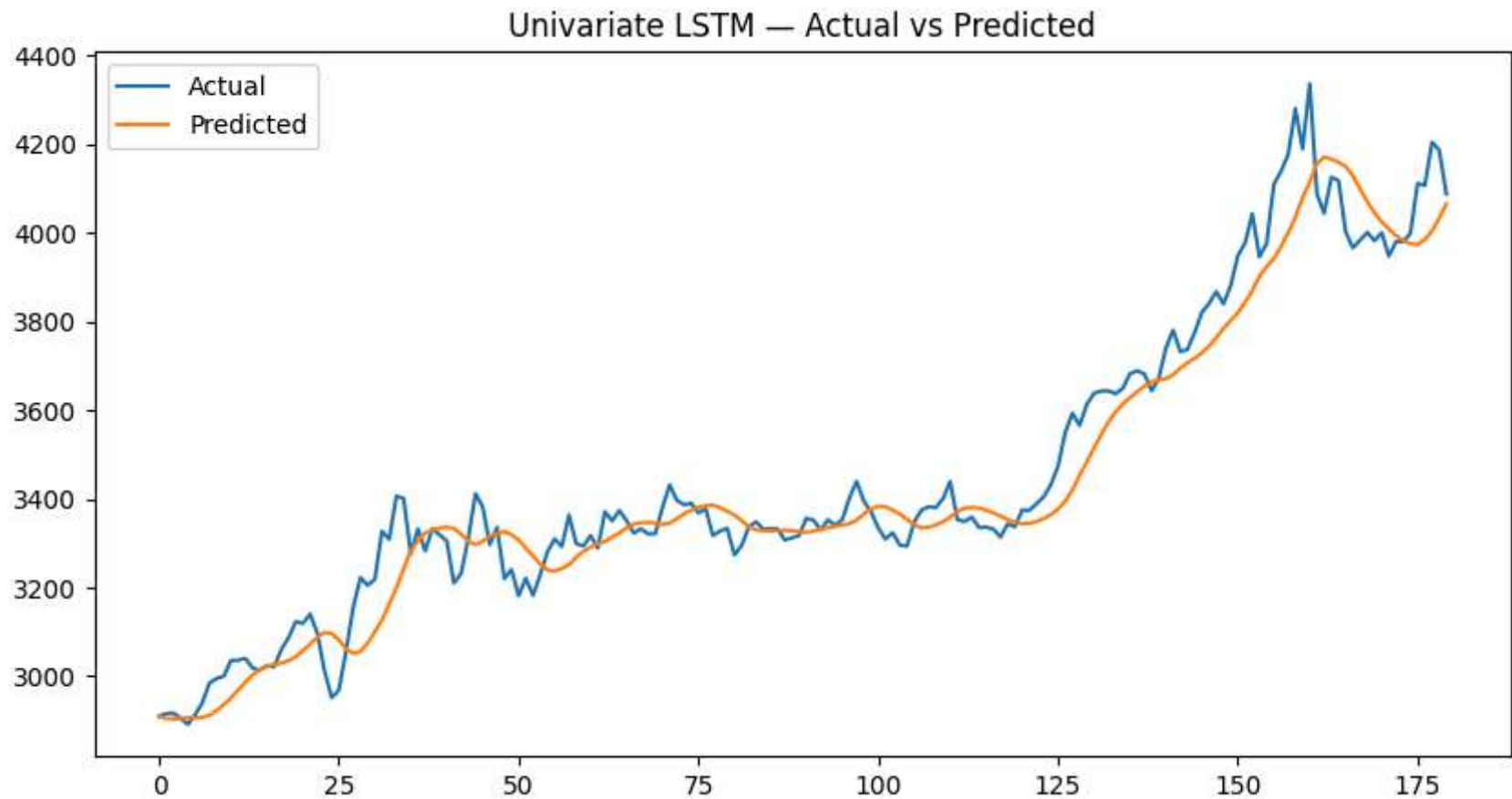
LSTM MAE: 61.81028747558594

LSTM MAPE: 1.7481202

Graph-Actual Vs Predicted

```
In [ ]: import matplotlib.pyplot as plt

plt.figure(figsize=(10,5))
plt.plot(y_test_orig, label="Actual")
plt.plot(pred, label="Predicted")
plt.title("Univariate LSTM - Actual vs Predicted")
plt.legend()
plt.show()
```



Comparison of the Metrics of all Three Models

```
In [ ]: import matplotlib.pyplot as plt

models = ["RNN", "GRU", "LSTM"]

rmse = [272.069384532696, 64.80756143793138, 81.15594014669382]
mae = [229.83074951171875, 49.33497619628906, 61.81028747558594]
mape = [6.356969, 1.4026474, 1.7481202]

colors = ["red", "green", "orange"]

plt.figure(figsize=(12,6))
```

```
# --- RMSE subplot ---
plt.subplot(1,3,1)
bars = plt.bar(models, rmse, color=colors)
plt.title("RMSE")
plt.ylabel("Error")
plt.grid(axis="y", linestyle="--", alpha=0.4)

# add value labels
for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, yval + 2, f"{yval:.2f}",
             ha='center', fontsize=9)

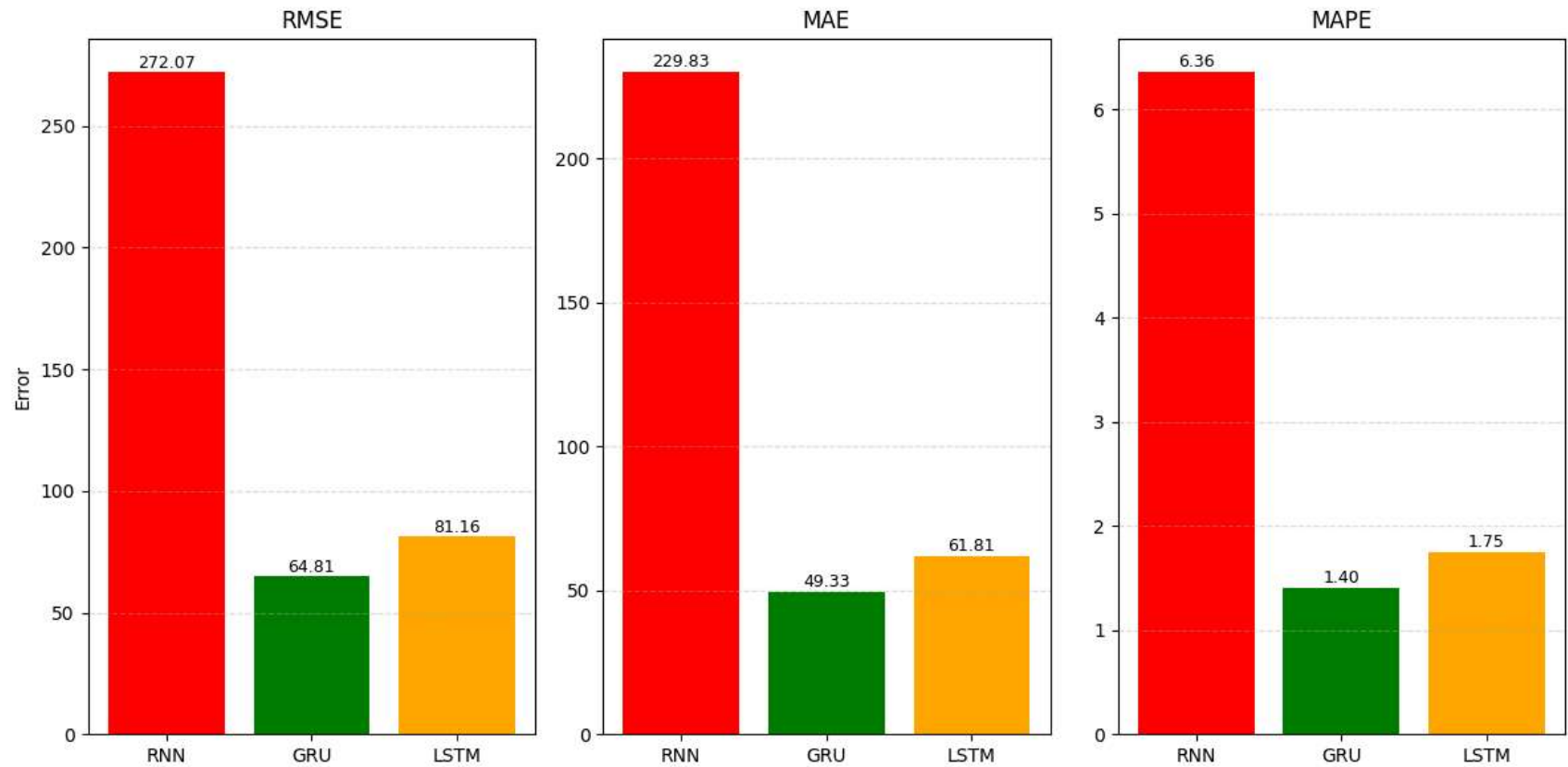
# --- MAE subplot ---
plt.subplot(1,3,2)
bars = plt.bar(models, mae, color=colors)
plt.title("MAE")
plt.grid(axis="y", linestyle="--", alpha=0.4)

for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, yval + 2, f"{yval:.2f}",
             ha='center', fontsize=9)

# --- MAPE subplot ---
plt.subplot(1,3,3)
bars = plt.bar(models, mape, color=colors)
plt.title("MAPE")
plt.grid(axis="y", linestyle="--", alpha=0.4)

for bar in bars:
    yval = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, yval + 0.05, f"{yval:.2f}",
             ha='center', fontsize=9)

plt.tight_layout()
plt.show()
```



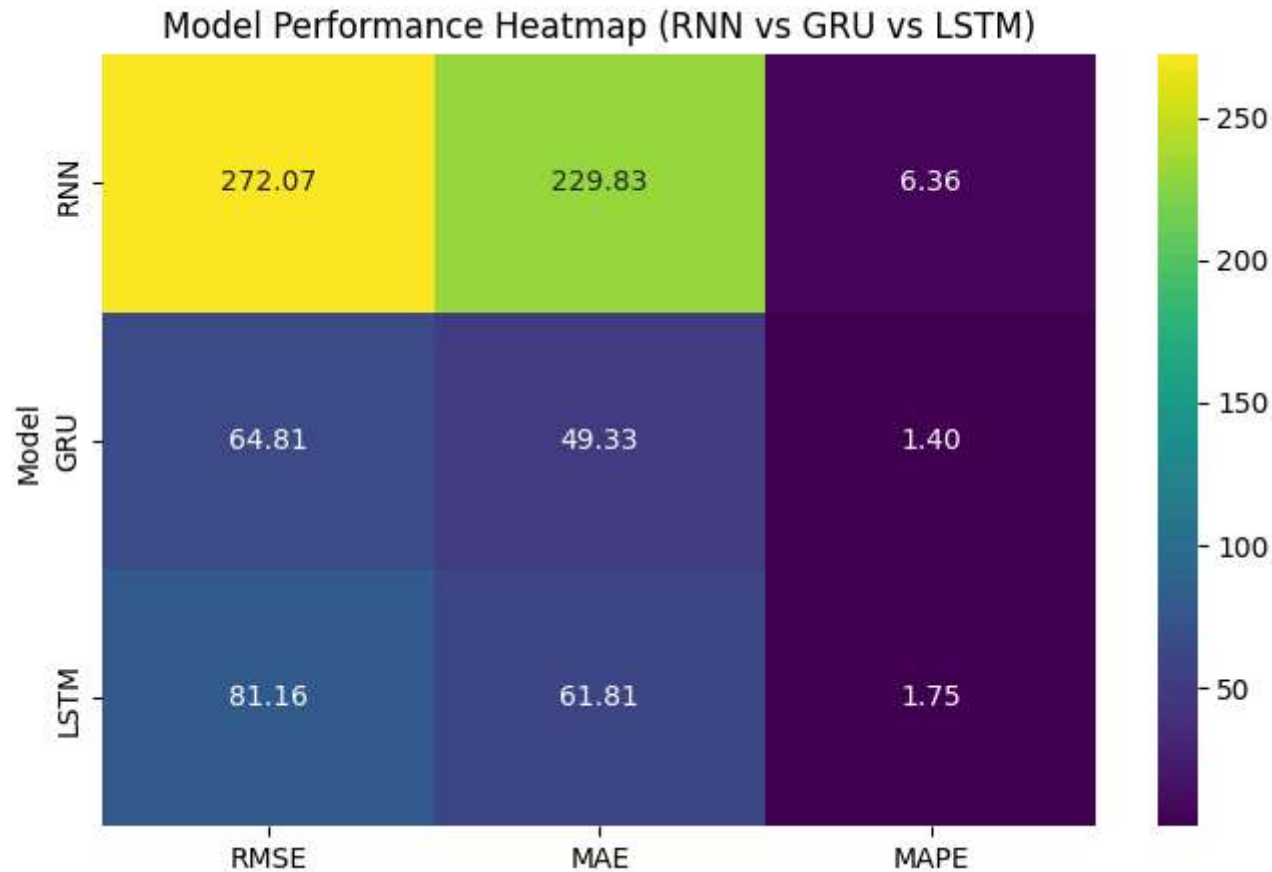
Confusion Matrix of the Metrics

```
In [ ]: import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

# Your model metrics
data = {
    "Model": ["RNN", "GRU", "LSTM"],
    "RMSE": [272.069384532696, 64.80756143793138, 81.15594014669382],
    "MAE": [229.83074951171875, 49.33497619628906, 61.81028747558594],
    "MAPE": [6.356969, 1.4026474, 1.7481202]
}

df = pd.DataFrame(data).set_index("Model")
```

```
plt.figure(figsize=(8,5))
sns.heatmap(df, annot=True, cmap="viridis", fmt=".2f")
plt.title("Model Performance Heatmap (RNN vs GRU vs LSTM)")
plt.show()
```



- 1.Strong accuracy: With 1.75% MAPE, LSTM provides reliable forecasts suitable for planning and decision-making.
 - 2.Moderate absolute errors: RMSE 81 and MAE 62 indicate stable predictions with manageable deviations.
 - 3.Consistent performance: LSTM captures patterns well, making it useful for demand forecasting, budgeting, and resource planning.
- Overall: A solid, accurate model—though GRU still performs better for this dataset.

GRU is the best performer with the lowest errors (MAPE ~1.4%). It delivers the most accurate and stable forecasts, making it ideal for budgeting, demand forecasting, and operational planning.

LSTM performs well (MAPE ~1.7%) and provides reliable, consistent predictions, but is slightly less accurate than GRU.

RNN shows weaker accuracy (MAPE ~6.3% and high absolute errors). It is not suitable for business-critical forecasting.

Overall Recommendation: Deploy GRU for business decision-making—it offers high precision, minimizes forecast risk, and supports more confident strategic planning.